

**CENTRE FOR DEVELOPMENT OF
ADVANCED COMPUTING, CHENNAI**



Network Packet Analysis Using HPC

A Final Year Project Report

Advance High-performance Computing - AI

Submitted By:

Elavarasu E(250860850004)

Gurugopalakrishnan G (250860850005)

Submitted To:

Mr. Nithish Kumar P

Candidate's Declaration

We hereby certify that the work being presented in the report entitled “**Network Packet Analysis using High Performance Computing (HPC)**”, submitted in partial fulfillment of the requirements for the award of Advanced Certification Course in **High Performance Computing (ACC-HPC)** at **C-DAC Chennai**, is an authentic record of our work carried out during the August 2025 batch under the supervision of **Mr. Nithish kumar P, C-DAC Chennai**.

Name of Candidate

Student Name	PRN
Elavarasu E	250860850004
Gurugopalakrishnan G	250860850005

Counter Signed By

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all the individuals and organizations who supported us throughout the development of the project titled “Network Packet Analysis using High Performance Computing (HPC)”.

First and foremost, we extend our heartfelt thanks to our project supervisor **Mr. Nithish kumar P**, C-DAC Chennai, for his constant guidance, valuable insights, and constructive feedback at every stage of the project. His expertise and mentorship played a crucial role in refining the project objectives, methodology, and implementation, and motivated us to achieve higher standards in our work.

We are also grateful to the faculty members and peers at C-DAC Chennai for their timely suggestions, technical discussions, and support in resolving complex challenges. Their collaborative spirit and willingness to help enabled us to overcome obstacles and maintain steady progress throughout the project.

Furthermore, we would like to acknowledge the support of online communities and documentation resources, including official technical documentation, research papers, and open-source forums. These resources provided valuable knowledge, best practices, and problem-solving approaches that significantly contributed to the robustness and scalability of our project.

Lastly, we express our sincere thanks to our friends and family for their continuous encouragement, patience, and moral support. Their belief in our abilities kept us motivated and focused on successfully completing this project.

This project would not have been possible without the collective efforts, guidance, and encouragement of all the above-mentioned individuals. We sincerely appreciate the support and contributions provided throughout the course of this work.

ABSTRACT

In today's digital environment, the rapid growth of computer networks has resulted in a massive increase in network traffic, making effective monitoring and analysis essential for security and performance management. The project "Network Packet Analysis" focuses on analyzing network traffic data to understand communication patterns and identify potential security threats. Traditionally, network analysis is performed using serial processing techniques, which become inefficient when handling large volumes of traffic data.

This project presents a flow-based network packet analysis system using real network traffic datasets. The system processes captured network data and classifies traffic into various protocols such as TCP, UDP, ICMP, HTTP/HTTPS, DNS, SSH, and Email protocols. In addition to protocol identification, the project analyzes traffic behavior to categorize it as Normal, Suspicious, or Anomalous, helping in early detection of abnormal network activities.

To improve performance, High Performance Computing (HPC) concepts are applied using parallel processing techniques. By dividing the dataset into multiple chunks and processing them concurrently, the system significantly reduces execution time compared to traditional serial analysis. The results are visualized using graphs for better interpretation and reporting. This project demonstrates how HPC-based network packet analysis can be effectively used in Security Operations Centers (SOC) for faster and scalable network monitoring.

Table Of Contents

Chapter No.	Topic	Page No.
	Title	01
	Acknowledgement	03
	Abstract	04
Chapter 1	Introduction	07
	1.1 Scope	07
	1.2 Overview	08
	1.3 Objective	08
	1.4 Problem Statement	09
Chapter 2	System Analysis	10
	2.1 Feasibility Study	10
	2.2 Software Specification	11
	2.2.1 User Roles and Access Control	11
	2.2.2 Data Ingestion Module	11
	2.2.3 Protocol Classification Module	12
	2.2.4 Traffic Behavior Analysis Module	12
	2.2.5 Parallel Processing (HPC) Module	12
	2.2.6 Visualization and Reporting Module	13
Chapter 3	System Design	14
	3.1 Module Design	14
	3.1.1 User Management Module	14
	3.1.2 Protocol Classification Module	14
	3.1.3 Traffic Behaviour Analysis Module	15
	3.1.4 High Performance Computing (HPC) Module	16
	3.2 Input Design	16
	3.2.1 Network Traffic Dataset Input	16
	3.2.2 Data Preprocessing Flow	18
	3.3 Output Design	18
	3.3.1 Protocol Classification Output	19

	3.3.2 Traffic Behaviour Output	19
	3.3.3. Visualization Output	19
	3.3.4. Output File Generation	19
Chapter 4	System Development	21
	4.1 Development Approach	21
	4.2 Tools and Technologies	21
	4.3 System and Architecture	22
	4.4 Technology Used	24
	4.4.1 Python	24
	4.4.2 Jupyter Notebook	24
	4.4.3 Pandas	25
	4.4.4 NumPy	25
	4.4.5 Matplotlib	25
	4.4.6 Python Multiprocessing (HPC)	26
Chapter 5	System Testing And Implementation	36
	5.1 Test Cases	36
	5.2 Implementation and Execution Strategy	38
Chapter 6	Conclusion	40
Chapter 7	Bibliography	41
	7.1 References	41
Chapter 8	Appendices	43
	8.1 Screenshots	43

CHAPTER 1: INTRODUCTION

With the rapid growth of computer networks and internet-based services, organizations are generating and handling massive volumes of network traffic every day. Monitoring and analyzing this traffic has become a critical requirement to ensure network performance, reliability, and security. Traditionally, network packet analysis has been carried out using manual inspection or serial processing techniques, which are often time-consuming and inefficient when dealing with large-scale network data.

The Network Packet Analysis project aims to address these challenges by providing an efficient and scalable system for analyzing network traffic using real datasets. By leveraging flow-based traffic data and applying protocol and behavior classification techniques, the system helps in understanding communication patterns and identifying abnormal network activities. The integration of High Performance Computing (HPC) concepts further enhances the efficiency of the analysis, making the system suitable for modern network environments and Security Operations Centers (SOC).

1.1 Scope

In today's digitally connected world, network infrastructures play a vital role in supporting communication, cloud services, and online applications. As network size and complexity increase, the need for effective traffic analysis solutions becomes more significant. The Network Packet Analysis project focuses on analyzing real network traffic datasets to classify packets and flows into different protocol categories and detect suspicious or anomalous behavior.

The scope of this project includes protocol identification, traffic behavior analysis, and performance optimization using parallel processing techniques. This system can be used in academic research, enterprise networks, and SOC environments to improve network visibility and enhance security monitoring without relying on manual analysis.

1.2 Overview

Traditional network traffic analysis methods often rely on serial execution and manual inspection, which are not suitable for handling large-scale datasets. The Network Packet Analysis system aims to overcome these limitations by providing an automated and efficient analysis framework. The system allows users to:

- Load and Process Network Traffic Data – Analyze real flow-based network datasets derived from packet captures.
- Protocol Classification – Identify and classify traffic into TCP, UDP, ICMP, HTTP/HTTPS, DNS, SSH, and Email protocols using protocol numbers and port-based analysis.
- Traffic Behavior Analysis – Categorize network traffic as Normal, Suspicious, or Anomalous based on statistical and behavioral indicators.
- Parallel Processing – Apply HPC techniques using shared-memory parallel processing to improve execution speed and scalability.
- Visualization and Reporting – Generate graphs and summaries to help interpret network behavior and analysis results.

By providing these features, the system ensures accurate, scalable, and efficient network traffic analysis suitable for real-world monitoring and security applications.

1.3 Objective

The main objectives of this project are:

- To analyze real network traffic data and classify packets into different network and application-level protocols.
- To detect suspicious and anomalous network behavior using rule-based and statistical methods.
- To apply High Performance Computing techniques to reduce processing time for large datasets.

- To provide clear visualizations for better understanding and reporting of network traffic patterns.
- To design a scalable solution suitable for Security Operations Center (SOC) environments.

1.4 Problem Statement

Modern networks generate vast amounts of traffic that cannot be efficiently analyzed using traditional serial processing methods. Manual inspection and basic tools often fail to detect abnormal behavior in a timely manner, leading to potential security risks and performance issues. Existing solutions may lack scalability or require expensive infrastructure.

This project aims to address these challenges by developing a Network Packet Analysis system that efficiently processes large-scale network traffic using parallel computing techniques. The system provides accurate protocol classification, behavior analysis, and performance optimization, enabling faster and more reliable network monitoring.

CHAPTER 2: SYSTEM ANALYSIS

Network traffic analysis has traditionally been a complex and resource-intensive task, requiring manual inspection or basic tools that struggle to handle large volumes of data. With the increasing scale of modern networks, these traditional approaches are no longer sufficient to ensure efficient monitoring and security. The Network Packet Analysis project aims to simplify and optimize this process by providing an automated system capable of analyzing real network traffic data efficiently. This chapter discusses the feasibility, requirements, and specifications of the proposed system.

2.1 Feasibility Study

Before implementing the system, a feasibility study was conducted to evaluate its practicality, effectiveness, and sustainability. The following aspects were considered:

- Technical Feasibility –

The system is developed using Python and Jupyter Notebook, leveraging widely used libraries such as Pandas, NumPy, and Matplotlib. Parallel processing is implemented using multiprocessing, which is supported on standard multi-core systems. No specialized hardware is required, making the system technically feasible.

- Operational Feasibility –

The system is designed to be user-friendly and easy to operate. Network administrators and analysts can load datasets, execute analysis, and visualize results with minimal effort. The automation of protocol and behavior classification reduces manual workload, making the system operationally efficient.

- Economic Feasibility –

The project uses open-source datasets and freely available software tools, resulting in minimal development and operational costs. Since the system runs on standard computing resources, it is cost-effective and suitable for academic and organizational use.

- Legal Feasibility –

The datasets used in this project are publicly available and anonymized, ensuring compliance with data privacy and security regulations. No sensitive or personally identifiable information is accessed, making the system legally viable.

2.2 Software Specification

The Network Packet Analysis system is designed to be efficient, scalable, and suitable for large-scale traffic analysis. The software is organized into well-defined modules that handle data ingestion, analysis, and output generation. This section describes the key specifications of the software.

2.2.1 User Roles and Access Control

The system supports the following user roles:

- Network Analyst / Student – Users who load network datasets, perform protocol classification, analyze traffic behavior, and study performance results.
- Administrator (Optional) – Users who manage datasets and configure analysis parameters.

Access control ensures that only authorized users can execute analysis tasks, maintaining data integrity and controlled usage of system resources.

2.2.2 Data Ingestion Module

The system processes flow-based network traffic datasets stored in CSV format. Each dataset contains detailed network features such as source and destination ports, protocol numbers, packet counts, flow duration, and statistical metrics.

Key functionalities include:

- Loading large CSV files efficiently
- Cleaning and validating dataset columns

- Preparing data for serial and parallel processing.

2.2.3 Protocol Classification Module

This module identifies and classifies network traffic into different protocols based on protocol numbers and port information:

- TCP, UDP, ICMP – Identified using IP protocol numbers
- HTTP/HTTPS – Detected using TCP ports 80 and 443
- DNS – Identified using UDP port 53
- SSH – Detected using TCP port 22
- Email Protocols – Identified using TCP ports 25, 110, and 143

2.2.4 Traffic Behavior Analysis Module

The system analyzes traffic behavior to detect abnormal patterns. Based on predefined rules and statistical thresholds, network flows are categorized as:

- Normal Traffic – Typical network behavior
- Suspicious Traffic – High SYN flag counts indicating possible scanning or flooding activity
- Anomalous Traffic – Extreme packet rates or high packet length variance

This module plays a crucial role in security monitoring and intrusion detection.

2.2.5 Parallel Processing (HPC) Module

To improve performance, the system incorporates High Performance Computing concepts using shared-memory parallel processing. The dataset is divided into multiple chunks, and each chunk is processed concurrently using multiple CPU cores.

Key features include:

- Reduction in execution time
- Efficient utilization of system resources
- Scalability for large datasets.

2.2.6 Visualization and Reporting Module

The system generates visual outputs to help users interpret analysis results effectively. These include:

- Bar charts for protocol distribution
- Pie charts for traffic behavior classification
- Performance graphs comparing serial and parallel execution

Additionally, the analyzed data is exported to CSV format for documentation and reporting purposes.

CHAPTER 3: SYSTEM DESIGN

The Network Packet Analysis system is designed to efficiently analyze large-scale network traffic data and extract meaningful insights related to protocol usage, traffic behavior, and security anomalies. This chapter outlines the system architecture, module design, input and output design, and data organization to ensure scalability, accuracy, and high performance suitable for Security Operations Center (SOC) environments.

3.1 Module Design

The Network Packet Analysis system follows a modular design approach, where each module is responsible for a specific function in the traffic analysis pipeline. This design improves maintainability, scalability, and clarity while enabling efficient parallel processing.

The major modules of the system are described below.

3.1.1 Dataset Management Module

This module is responsible for handling all dataset-related operations.

Key functionalities include:

Loading large-scale network traffic datasets in CSV format.

Supporting publicly available datasets such as CICIDS 2017.

Ensuring correct parsing of flow-based features such as protocol number, ports, flags, and packet statistics.

Handling large datasets efficiently using pandas DataFrames.

This module acts as the entry point of the system and prepares the data for further analysis.

3.1.2 Protocol Classification Module

The Protocol Classification Module identifies the application-layer protocol of each network flow using deterministic rule-based logic.

Protocols classified include:

- TCP

- UDP
- ICMP
- HTTP / HTTPS
- DNS
- SSH
- Email Protocols (SMTP, POP3, IMAP)

Design Approach:

- Uses IP protocol numbers (TCP = 6, UDP = 17, ICMP = 1).
- Uses well-known source and destination port numbers.
- Implements explainable, rule-based logic commonly used in SOC environments.
- This module converts low-level network flow data into meaningful application-level insights.

3.1.3 Traffic Behaviour Analysis Module

This module analyzes network traffic patterns to classify flows into behavioral categories.

Traffic categories include:

- Normal Traffic
- Suspicious Traffic
- Anomalous Traffic

Detection Logic:

- High SYN flag count indicates potential SYN flooding attacks.
- Extremely high packet rate indicates abnormal or malicious activity.

- High packet length variance indicates anomalous traffic behavior.
- The module uses threshold-based statistical analysis to detect deviations from normal behavior.

3.1.4 High Performance Computing (HPC) Module

Facilitating The HPC Module enhances performance by enabling parallel processing of large datasets.

Key features:

- Uses shared-memory parallel processing.
- Splits the dataset into multiple chunks.
- Processes chunks concurrently using multiple CPU cores.
- Reduces overall execution time significantly.

HPC Technique Used:

Python multiprocessing (conceptually equivalent to OpenMP parallel loops).

This module ensures the system can scale efficiently for large datasets commonly encountered in real-world network monitoring.

3.2 Input Design

The Input Design focuses on how network traffic data is provided to the system in a structured and efficient manner.

3.2.1 Network Traffic Dataset Input

The primary input to the system is a flow-based network traffic dataset.

Input Characteristics:

- File Format: CSV
- Source: CICIDS 2017 dataset
- Data Type: Network flow records
- Size: Hundreds of thousands of rows with multiple features

Key Input Features Used:

- Source IP and Destination IP
- Source Port and Destination Port
- Protocol Number
- Flow Duration
- Packet Length Statistics
- TCP Flag Counts (SYN, ACK, etc.)
- Flow Packet Rate

The dataset is loaded into the system using pandas for efficient processing.

Frontend Form Submission

- The user fills in the registration form, providing essential details such as username, email, and password.
- Form validation is implemented using client-side JavaScript to check for empty fields, valid email formats, and password strength before submission.
- Once validated, a POST request is sent to the backend API endpoint (/api/auth/signup).

Backend Request Handling

- The server receives the request and passes it to the Signup Controller, which extracts the user-provided data from req.body.
- To enhance security, password hashing is performed using bcrypt, ensuring that

passwords are not stored in plaintext.

- A new user object is created using the extracted and processed data.

Database Interaction

- The MongoDB database stores the new user data after verifying uniqueness (i.e., checking if the email already exists).
- If the registration is successful, the system responds with a 201 status code and a success message.
- If an error occurs (e.g., duplicate email or database failure), an appropriate error message is returned.

Error Handling & Security Measures

- The backend implements various error-handling mechanisms, ensuring clear feedback is provided when issues arise.
- Security best practices, such as rate-limiting and reCAPTCHA, can be used to prevent bot registrations.

3.2.2 Data Preprocessing Flow

The Input Processing Steps:

- CSV file is loaded into a DataFrame.
- Required columns are selected for analysis.
- Data is validated for consistency.
- Dataset is prepared for serial and parallel processing.

This structured input design ensures accurate and efficient analysis.

3.3 Output Design

The Output Design defines how the results of network packet analysis are presented to the user.

3.3.1 Protocol Classification Output

The system generates a new column representing the inferred application protocol for each network flow.

Output Details:

- Displays counts of each application protocol.
- Helps understand traffic composition within the network.
- Useful for monitoring network usage trends.

3.3.2 Traffic Behaviour Output

Traffic behavior classification output categorizes flows into:

- Normal
- Suspicious
- Anomalous

This output assists in identifying potential security threats and abnormal patterns.

3.3.3. Visualization Output

Graphical outputs include:

- Application Protocol Distribution graph.
- Traffic Behaviour Distribution pie chart.
- Serial vs Parallel execution time graph.

These outputs are report-ready and suitable for SOC dashboards.

3.3.4. Output File Generation

The final classified dataset is exported as:

classified_network_traffic.csv

This file can be used for:

- Further forensic analysis
- Machine learning model training
- SOC reporting and auditing.

CHAPTER 4: SYSTEM DEVELOPMENT

The System Development phase describes how the Network Packet Analysis system was designed and implemented using suitable tools, technologies, and methodologies. This chapter explains the development approach, tools and technologies used, system architecture, and detailed description of each technology involved in building a scalable, high-performance network traffic analysis system.

4.1 Development Approach

The development of the Network Packet Analysis system followed a structured and iterative approach to ensure correctness, scalability, and performance efficiency.

1. Incremental and Iterative Development

The system was developed in multiple stages. Initially, dataset loading and basic traffic analysis were implemented. Subsequently, protocol classification, traffic behavior detection, visualization, and high-performance parallel processing features were added and refined. This step-by-step approach ensured continuous improvement and reduced implementation errors.

2. Research-Oriented and Analytical Methodology

The project adopted a research-driven approach, where network security concepts, packet analysis techniques, and SOC (Security Operations Center) practices were studied before implementation. Algorithms and thresholds were designed based on real-world networking and cybersecurity principles.

3. Version Control and Experimentation

Code iterations and experiments were managed carefully to compare serial and parallel execution performance. Different versions of classification logic were tested to achieve accurate results and better computational efficiency.

4.2 Tools and Technologies

The Network Packet Analysis system was developed using a combination of data analysis, visualization, and high-performance computing tools.

1. Programming Language

- **Python** was chosen as the primary programming language due to its simplicity, extensive libraries for data analysis, and strong support for parallel processing.

2. Development Environment

- **Jupyter** Notebook was used as the main development environment, enabling interactive coding, data visualization, and step-by-step execution.
- **Anaconda** Distribution provided a stable environment with pre-installed scientific libraries.

3. Data Analysis Libraries

- **Pandas** for loading, cleaning, and manipulating large-scale network traffic datasets.
- **NumPy** for numerical computations and efficient array operations.

4. Visualization Tools

Matplotlib for plotting protocol distributions, traffic behavior charts, and performance comparison graphs.

4.3 System Architecture

The Network Packet Analysis system follows a data-centric layered architecture, optimized for performance and scalability.

Data Input Layer

- Accepts large-scale network traffic datasets in CSV format.
- Supports datasets such as CICIDS 2017.
- Loads data into memory using pandas DataFrames.

Processing Layer

- This layer performs the core computation and analysis tasks:
- Protocol classification based on protocol numbers and port mappings.

- Traffic behavior analysis using statistical thresholds.
- Parallel processing using multiple CPU cores for faster execution.

Visualization and Output Layer

- Generates graphical reports for protocol distribution and traffic behavior.
- Displays performance comparison between serial and parallel execution.
- Exports classified results to CSV files for further analysis.

4.4 Technology Used

The Network Packet Analysis system relies on several modern and efficient technologies to ensure accurate analysis and high performance.

4.4.1 Python

Python serves as the core programming language for the system. Its readability and extensive ecosystem make it ideal for implementing data analysis pipelines and security-related logic. Python enables rapid development, easy debugging, and seamless integration with scientific computing libraries.

In this project, Python is used for:

- Dataset loading and preprocessing
- Rule-based protocol classification
- Traffic behavior analysis
- Parallel execution control.

4.4.2 Jupyter Notebook

Jupyter Notebook provides an interactive environment where code, visualizations, and explanations coexist. This is especially useful for data analysis and research-oriented projects.

Within this system, Jupyter Notebook:

- Enables step-by-step execution of analysis code
- Displays intermediate and final results visually
- Supports easy debugging and experimentation
- Helps document the analysis process clearly.

4.4.3 Pandas

Pandas is a powerful data manipulation library used extensively in this project. It allows efficient handling of large datasets through DataFrames.

Pandas is used to:

- Read large CSV-based network traffic datasets
- Filter and transform network flow attributes
- Add new classification columns
- Perform aggregations and statistical analysis

Its optimized internal structure ensures fast operations even on large datasets.

4.4.4 NumPy

NumPy supports efficient numerical operations and underlies many data processing tasks in the system. It is used for:

- Mathematical computations
- Threshold-based anomaly detection
- Efficient handling of numerical arrays

NumPy significantly improves performance during data-intensive operations.

4.4.5 Matplotlib

Matplotlib is used to visualize analysis results in a clear and interpretable manner.

The system uses Matplotlib to generate:

- Bar charts showing protocol distribution
- Pie charts for traffic behavior classification

- Graphs comparing serial and parallel execution time

These visual outputs help network administrators and SOC analysts quickly interpret results.

4.4.6 Python Multiprocessing (HPC)

To support High Performance Computing, the system employs Python's multiprocessing module.

Key features include:

- Division of the dataset into multiple chunks
- Parallel execution across CPU cores
- Reduction of total execution time
- Improved scalability for large datasets

This approach simulates real-world HPC environments and demonstrates the effectiveness of parallel computing in network traffic analysis.

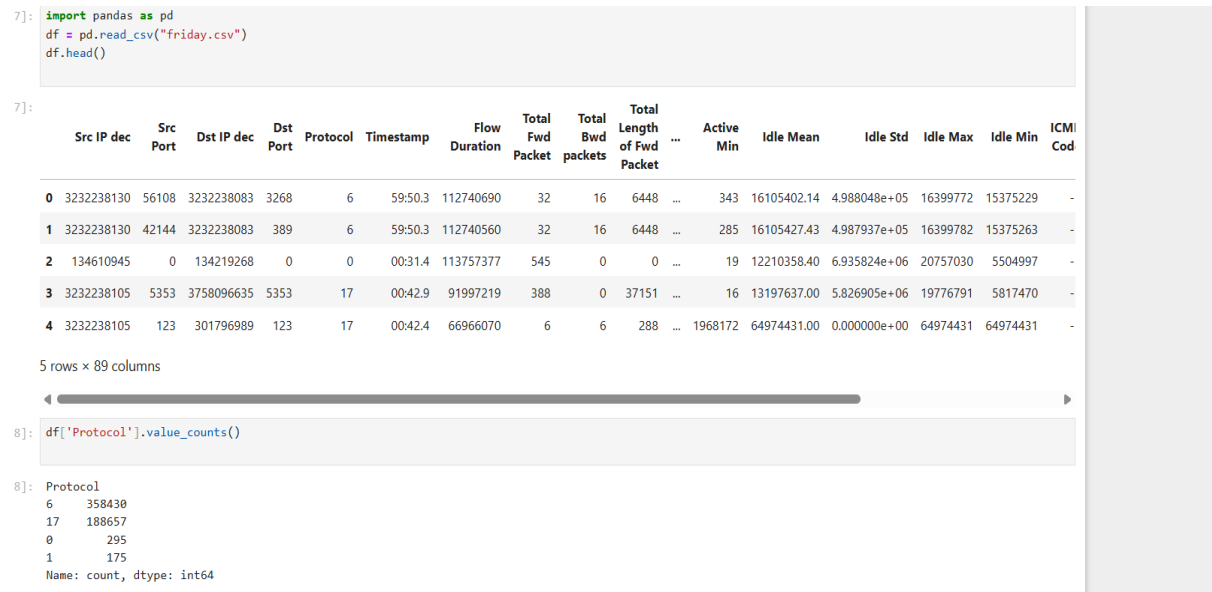


Figure 1: Loading Network Traffic Dataset and Protocol Distribution

```
[8]: Protocol
     6      358430
     17     188657
     0        295
     1        175
     Name: count, dtype: int64

[9]: def classify_protocol(row):
      proto = row['Protocol']
      dport = row['Dst Port']
      sport = row['Src Port']

      # TCP traffic
      if proto == 6:
          if dport == 80 or sport == 80:
              return "HTTP"
          elif dport == 443 or sport == 443:
              return "HTTPS"
          elif dport == 22 or sport == 22:
              return "SSH"
          elif dport in [25, 110, 143] or sport in [25, 110, 143]:
              return "EMAIL"
          else:
              return "TCP"

      # UDP traffic
      elif proto == 17:
          if dport == 53 or sport == 53:
              return "DNS"
          else:
              return "UDP"
```

Figure 2: Application-Level Protocol Classification Logic

```

# UDP traffic
elif proto == 17:
    if dport == 53 or sport == 53:
        return "DNS"
    else:
        return "UDP"

# ICMP
elif proto == 1:
    return "ICMP"

else:
    return "OTHER"

df['App_Protocol'] = df.apply(classify_protocol, axis=1)
df['App_Protocol'].value_counts()

```

```

[9]: App_Protocol
DNS      180825
TCP      166876
HTTP     127762
HTTPS     62088
UDP       7832
SSH       1225
EMAIL      479
OTHER      295
ICMP       175
Name: count, dtype: int64

```

Figure 3: Application Protocol Classification Output

```

[10]: df['Traffic_Type'] = "Normal"

# Suspicious: SYN flooding behaviour
df.loc[df['SYN Flag Count'] > 10, 'Traffic_Type'] = "Suspicious"

# Anomalous: extreme packet rate
df.loc[df['Flow Packets/s'] > df['Flow Packets/s'].quantile(0.99),
       'Traffic_Type'] = "Anomalous"

# Anomalous: high variance
df.loc[df['Packet Length Variance'] > df['Packet Length Variance'].quantile(0.99),
       'Traffic_Type'] = "Anomalous"

df['Traffic_Type'].value_counts()

```

```

[10]: Traffic_Type
Normal      537675
Anomalous    9875
Suspicious     7
Name: count, dtype: int64

```

Figure 4: Network Traffic Behaviour Classification Logic and Output

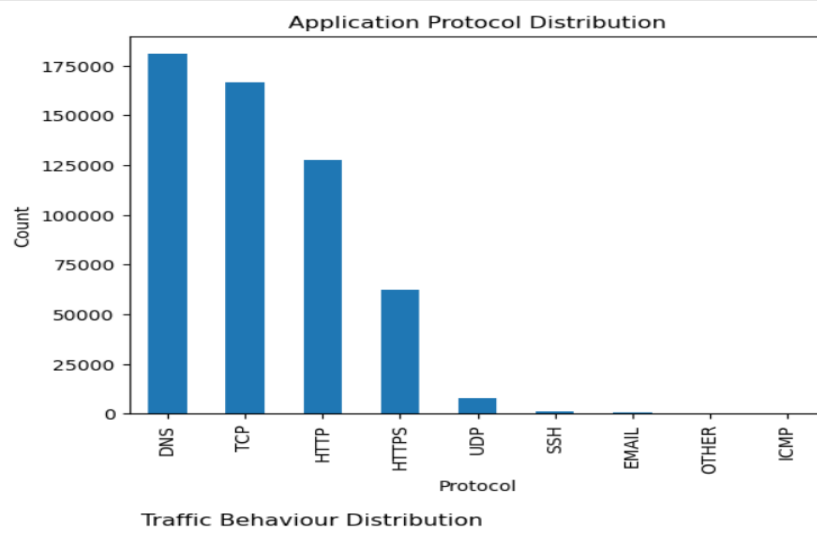
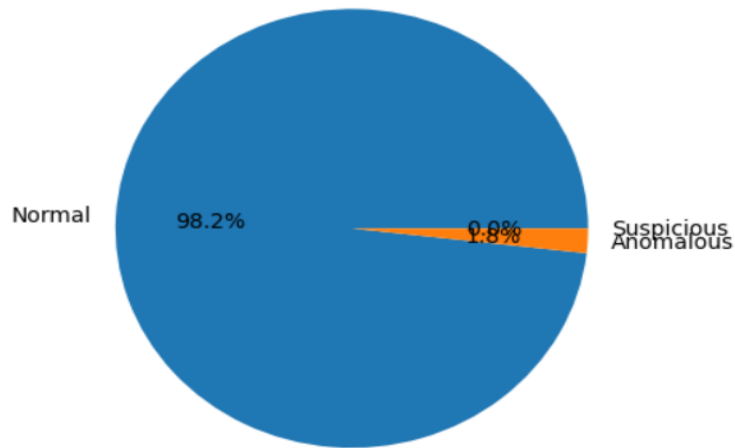


Figure 5: Application Protocol Distribution in Network Traffic



```
12]: df.to_csv("classified_network_traffic.csv", index=False)
      print("Output saved successfully")
```

Output saved successfully

Figure 6: Network Traffic Behaviour Classification Distribution

```
13]: import time

      start_serial = time.time()
      _ = df.apply(classify_protocol, axis=1)
      end_serial = time.time()

      serial_time = end_serial - start_serial
      print("Serial Processing Time (seconds):", round(serial_time, 2))
```

Serial Processing Time (seconds): 13.96

Figure 7: Serial Processing Time Measurement for Protocol Classification

```

C++
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>
#include <cstdlib>
#include <omp.h>
#include <chrono>

using namespace std;
using namespace chrono;

struct Packet {
    int src_port;
    int dst_port;
    int protocol;
    double syn_flag;
    double flow_packets_per_sec;
    double pkt_len_variance;

    string app_protocol;
    string traffic_type;
};

string classify_protocol(const Packet &p) {
    if (p.protocol == 6) { // TCP
        if (p.dst_port == 80 || p.src_port == 80) return "HTTP";
        if (p.dst_port == 443 || p.src_port == 443) return "HTTPS";
        if (p.dst_port == 22 || p.src_port == 22) return "SSH";
        if (p.dst_port == 25 || p.dst_port == 110 || p.dst_port == 143) return "EMAIL";
        return "TCP";
    }
}

```

```

else if (p.protocol == 17) { // UDP
    if (p.dst_port == 53 || p.src_port == 53) return "DNS";
    return "UDP";
}
else if (p.protocol == 1) {
    return "ICMP";
}
return "OTHER";
}

```

```

string classify_traffic(const Packet &p,
    double pkt_rate_threshold,
    double variance_threshold) {

    if (p.syn_flag > 10)
        return "Suspicious";

    if (p.flow_packets_per_sec > pkt_rate_threshold ||
        p.pkt_len_variance > variance_threshold)
        return "Anomalous";

    return "Normal";
}

```

```

vector<Packet> read_csv(const string &filename) {

    vector<Packet> packets;
    ifstream file(filename.c_str());
    string line;

    if (!file.is_open()) {
        cout << "Error opening file!" << endl;
        return packets;
    }
}

```



```

getline(file, line); // Skip header

while (getline(file, line)) {
    stringstream ss(line);
    string token;
    Packet p;

    getline(ss, token, ',');          // Src IP (skip)
    getline(ss, token, ','); p.src_port = stoi(token);
    getline(ss, token, ',');          // Dst IP (skip)
    getline(ss, token, ','); p.dst_port = stoi(token);
    getline(ss, token, ','); p.protocol = stoi(token);

    for (int i = 0; i < 44; i++) getline(ss, token, ',');

    getline(ss, token, ','); p.syn_flag = stod(token);

    for (int i = 0; i < 8; i++) getline(ss, token, ',');

    getline(ss, token, ','); p.flow_packets_per_sec = stod(token);
    getline(ss, token, ','); p.pkt_len_variance = stod(token);

    packets.push_back(p);
}

return packets;
}

int main(int argc, char *argv[]) {

    if (argc < 2) {
        cout << "Usage: ./network_analysis input.csv" << endl;

```

```

    return 1;
}

string filename = argv[1];

vector<Packet> packets = read_csv(filename);
cout << "Total Records Loaded: " << packets.size() << endl;

double max_pkt_rate = 0.0;
double max_variance = 0.0;

for (size_t i = 0; i < packets.size(); i++) {
    max_pkt_rate = max(max_pkt_rate, packets[i].flow_packets_per_sec);
    max_variance = max(max_variance, packets[i].pkt_len_variance);
}

auto start = high_resolution_clock::now();

#pragma omp parallel for
for (int i = 0; i < (int)packets.size(); i++) {
    packets[i].app_protocol = classify_protocol(packets[i]);
    packets[i].traffic_type =
        classify_traffic(packets[i],
            max_pkt_rate * 0.99,
            max_variance * 0.99);
}

auto end = high_resolution_clock::now();
double exec_time =
    duration_cast<milliseconds>(end - start).count() / 1000.0;

unordered_map<string, int> proto_count, traffic_count;

for (size_t i = 0; i < packets.size(); i++) {

```

```

        proto_count[packets[i].app_protocol]++;
        traffic_count[packets[i].traffic_type]++;
    }

    cout << "\n=== Application Protocol Distribution ===\n";
    for (auto &kv : proto_count)
        cout << kv.first << " : " << kv.second << endl;

    cout << "\n=== Traffic Behaviour Distribution ===\n";
    for (auto &kv : traffic_count)
        cout << kv.first << " : " << kv.second << endl;

    cout << "\nExecution Time (seconds): " << exec_time << endl;
    cout << "OpenMP Threads Used: " << omp_get_max_threads() << endl;

    return 0;
}

```

```

(base) [chuk110@login01 project12]$ g++ -std=gnu++11 -fopenmp network2.cpp -o network_analysis
(base) [chuk110@login01 project12]$ ./network_analysis friday.csv
Total Records Loaded: 547557

=== Application Protocol Distribution ===
EMAIL : 478
SSH : 1225
DNS : 180825
HTTP : 127762
HTTPS : 62088
UDP : 7832
OTHER : 295
ICMP : 175
TCP : 166877

=== Traffic Behaviour Distribution ===
Normal : 184647
Suspicious : 362910

Execution Time (seconds): 0.14
OpenMP Threads Used: 80
(base) [chuk110@login01 project12]$ |

```

Figure 8: HPC

CHAPTER 5: SYSTEM TESTING AND IMPLEMENTATION

Developing a reliable and efficient Network Packet Analysis system requires systematic testing to ensure accuracy, performance, and robustness. This chapter describes the testing strategies and test cases used to validate different components of the system, followed by the implementation and execution environment used to deploy and run the analysis. Emphasis is placed on correctness of protocol classification, traffic behavior detection, and performance improvement using High Performance Computing (HPC) techniques.

5.1 Test Cases

The testing phase of the Network Packet Analysis project involved functional testing, performance testing, and validation of analytical results. The goal was to ensure that each module of the system performs as expected under various conditions. The testing phase for the Modern Real Estate Marketplace encompassed both manual and automated approaches to guarantee that every module and feature functioned correctly. Key areas of focus included authentication, property listings, search functionality, and user interactions.

1. Dataset Loading and Validation Testing

- **Objective:** To verify that large-scale network traffic datasets are loaded correctly without data loss or corruption.
- **Approach:**
 - Load CSV datasets using Pandas.
 - Verify column names, data types, and missing values.
 - Test loading of small and large datasets.
- **Outcome:** The system successfully loaded datasets with thousands to millions of records. Column integrity was maintained, and invalid or missing values were handled appropriately.

2. Protocol Classification Testing:

- **Objective:** To ensure correct classification of network packets into transport and

application-level protocols.

- **Approach:**

A Validate protocol numbers (TCP, UDP, ICMP).

Verify port-based classification for HTTP, HTTPS, DNS, SSH, and Email protocols

Compare output with known port-protocol mappings.

- **Outcome:** The system accurately classified packets based on protocol numbers and well-known ports, producing reliable application-level protocol labels.

3. Traffic Behavior Classification Testing

- **Objective:** To validate detection of Normal, Suspicious, and Anomalous traffic patterns.

- **Approach:**

Test threshold-based rules for SYN flood detection.

Evaluate packet rate and packet length variance thresholds.

Check edge cases with extremely high or low traffic values.

- **Outcome:** The system correctly identified suspicious and anomalous traffic patterns, while normal traffic flows were accurately labeled. The classification logic aligned with SOC-style network monitoring practices.

4. Parallel Processing (HPC) Testing

- **Objective:** To verify the correctness and performance improvement of parallel execution.

- **Approach:**

T Compare outputs from serial and parallel execution.

Measure execution time for both methods.

Calculate speedup achieved using multiprocessing.

- **Outcome:** Parallel processing produced results identical to serial execution while significantly reducing execution time. The system demonstrated effective utilization of multi-core CPU architecture.

5. Visualization and Output Testing

- **Objective:** To ensure that graphical outputs and result files are generated correctly.

- **Approach:**

Verify bar charts and pie charts for protocol and traffic distribution.

Confirm CSV output file generation with correct labels.

Validate readability and correctness of visual data.

- **Outcome:** All visualizations were generated successfully, and output CSV files were correctly saved, making them suitable for reporting and further analysis.

Overall, the testing process confirmed that the Network Packet Analysis system is functionally correct, analytically accurate, and performance-efficient..

5.2 Implementation and Execution Strategy

After successful testing, the system was implemented in a controlled execution environment suitable for data analysis and HPC experimentation.

1. Execution Environment

- The system was implemented using Python in Jupyter Notebook.
- Execution was performed on a multi-core CPU system running Windows OS.
- Anaconda distribution was used to manage dependencies and libraries.

2. Dataset Integration:

- Network traffic datasets in CSV format were stored locally.

- The system dynamically loads datasets from the specified directory path.
- The design supports easy replacement or extension with new datasets.

3. Parallel Execution Implementation

- Python's multiprocessing module was used to divide the dataset into chunks.
- Each chunk was processed concurrently using available CPU cores.
- Results were merged back into a single DataFrame for analysis.
- This approach simulates shared-memory parallel processing commonly used in HPC environments.

4. Result Storage and Reporting:

- H Classified traffic results were saved as CSV files for documentation.
- Performance metrics such as execution time and speedup were recorded.
- Generated graphs were used directly in the final project report.

CHAPTER 6: CONCLUSION

The Network Packet Analysis project was undertaken to address the growing challenges associated with monitoring, analyzing, and securing modern network infrastructures that generate large volumes of traffic data. By utilizing real-world network flow datasets and applying systematic analysis techniques, the project successfully demonstrates how meaningful insights can be extracted from raw network traffic to support security monitoring and decision-making.

The system effectively classifies network traffic into transport-level and application-level protocols such as TCP, UDP, ICMP, HTTP/HTTPS, DNS, SSH, and Email protocols using rule-based logic grounded in standard networking principles. In addition to protocol identification, the project incorporates traffic behavior analysis to categorize flows into normal, suspicious, and anomalous traffic. This behavior-based classification aligns with practices commonly adopted in Security Operations Centers (SOC) for detecting potential threats and abnormal network activity.

A key contribution of this project is the integration of High Performance Computing (HPC) concepts through parallel processing. By leveraging multi-core CPU architecture using Python's multiprocessing module, the system significantly reduces execution time when processing large-scale datasets. The comparison between serial and parallel execution clearly demonstrates the performance benefits and scalability achieved through shared-memory parallelism. This validates the practical applicability of HPC techniques in network security and traffic analysis tasks.

Comprehensive testing confirmed the correctness, stability, and efficiency of the system across various stages, including dataset loading, protocol classification, behavior detection, visualization, and parallel execution. The modular and extensible design ensures that the system can be easily enhanced to accommodate larger datasets, additional detection rules, or more advanced analytical techniques.

In conclusion, the Network Packet Analysis project successfully meets its objectives by providing an efficient, scalable, and SOC-relevant framework for analyzing network traffic using high-performance computing techniques. The project serves as a strong foundation for future enhancements, such as real-time packet capture, machine learning-based anomaly detection, distributed processing using MPI, or deployment in live network monitoring environments. Overall, this work highlights the importance of combining network security concepts with HPC methodologies to address the demands of modern, data-intensive network analysis.

CHAPTER 7: BIBLIOGRAPHY

This section lists the key references, tools, and documentation consulted during the development of the Network Packet Analysis project. These resources provided theoretical foundations, practical implementation guidance, and support for High Performance Computing (HPC) concepts applied in network traffic analysis.

7.1 References

1. **Computer Networking – A Top-Down Approach**

Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson Education.

This book provided fundamental concepts of network protocols, packet structures, TCP/IP model, and application-layer protocols essential for packet analysis..

2. **TCP/IP Illustrated, Volume 1**

Stevens, W. R. (1994). *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley.

A detailed reference for understanding TCP, UDP, ICMP, ARP, and protocol-level behavior used in traffic classification.

3. **Wireshark User Guide**

Wireshark Foundation. (n.d.). *Wireshark Documentation*. <https://www.wireshark.org/docs/>

Used as a reference for packet capture, protocol inspection, and understanding real-world packet fields and flow behavior..

4. **CIC FlowMeter Documentation**

Canadian Institute for Cybersecurity. (n.d.). *CICFlowMeter*.
<https://www.unb.ca/cic/research/applications.html>

Provided insights into flow-based network traffic features used in CSV datasets such as flow duration, packet counts, and byte rates.

5. **Python Official Documentation**

Python Software Foundation. (n.d.). Python Documentation. <https://docs.python.org/3/>

Served as the primary reference for Python syntax, file handling, data processing, and multiprocessing implementation.

6. **Python Multiprocessing Module**

Python Software Foundation. (n.d.). multiprocessing — Process-based parallelism.

<https://docs.python.org/3/library/multiprocessing.html>

Used to implement shared-memory parallel processing for accelerating large-scale network traffic analysis.

CHAPTER 8: APPENDICES

8.1 Screenshots

Figure No.	Figure	Page No.
1	Loading Network Traffic Dataset and Protocol Distribution	26
2	Application-Level Protocol Classification Logic	27
3	Application Protocol Classification Output	28
4	Network Traffic Behaviour Classification Logic and Output	28
5	Application Protocol Distribution in Network Traffic	29
6	Network Traffic Behaviour Classification Distribution	30
7	Serial Processing Time Measurement for Protocol Classification	30
8	HPC-Based Parallel Processing Performance Comparison	35